INFORME DE DESARROLLO SEGURO

Protecciones frente a vulnerabilidades

DevOps, DevSecops

Estudiantes: Víctor Contreras, Fabian Gacitua, Sebastián Carrillo, José Óses

Asignatura: Desarrollo Avanzado

Docente: César Varela

Institución: Universidad Católica de la Santísima Concepción

Fecha: 29 de octubre de 2026

1. Objetivo

Este informe resume las principales protecciones de seguridad del proyecto Gestor de Productos y explica qué vulnerabilidades ayudan a prevenir. Se enfoca en las medidas de seguridad.

2. Protecciones frente a vulnerabilidades

- **Contraseñas protegidas:** Se utiliza Argon2id para almacenar contraseñas como hashes y no como texto legible. Esto reduce el impacto si alguien accede a la base de datos.
- **Sesiones y autenticación:** Las sesiones permiten mantener el inicio de sesión y controlar el acceso a funciones privadas. Un secreto de sesión seguro y cookies bien configuradas ayudan a proteger las sesiones.
- **Control de acceso:** Las operaciones deben comprobar que el usuario tenga permiso para realizarlas. Esto ayuda a evitar que una persona consulte o modifique información ajena.
- **Consultas SQL parametrizadas:** Separar los datos de las instrucciones SQL reduce el riesgo de inyección SQL, que puede permitir alterar consultas o acceder a información no autorizada.
- **Validación de datos:** Revisar los datos recibidos antes de procesarlos ayuda a rechazar entradas incorrectas o inesperadas.
- **Protección contra XSS:** El escape de contenido y la política de seguridad de contenido (CSP), cuando se aplican correctamente, ayudan a evitar que se ejecuten scripts maliciosos en el navegador.
- **Cabeceras de seguridad:** Helmet configura cabeceras HTTP que agregan restricciones de seguridad al navegador y ayudan a reducir ciertos ataques web.
- **Protección CSRF y origen:** Comprobar el origen de las solicitudes ayuda a rechazar peticiones externas que intenten realizar acciones usando la sesión de una persona.
- **Límite de solicitudes:** Limitar los intentos o solicitudes dificulta el abuso automatizado y ayuda a reducir ataques de fuerza bruta y sobrecarga.
- **Manejo de secretos:** Las claves deben mantenerse fuera del código fuente y configurarse mediante variables de entorno. Un secreto débil o expuesto puede comprometer las sesiones.

3. Recomendaciones

- **Dependencias:** Ejecutar periódicamente npm audit y actualizar paquetes vulnerables.
- **Revisión de código:** Usar herramientas como ESLint y SonarQube para detectar errores y posibles problemas de seguridad.
- **Pruebas de seguridad:** Probar los permisos, el inicio de sesión, los formularios y el envío de datos maliciosos.
- **Configuración de producción:** Usar HTTPS, secretos fuertes y únicos, y evitar mostrar errores con información interna.

4. Conclusión

Las medidas descritas ayudan a reducir riesgos como la inyección SQL, el abuso de sesiones, el acceso no autorizado y la ejecución de scripts maliciosos. La seguridad requiere combinar controles en el código con una configuración segura y revisiones periódicas. Las recomendaciones deben verificarse antes de considerarlas implementadas.